Objašnjenje recommender sistema

Ovaj recommender sistem, u sklopu aplikacije „StoneCarve Manager", razvijen je kako bi korisnicima mobilne aplikacije preporučio kamene proizvode na osnovu njihove sličnosti sa trenutno odabranim proizvodom. Sličnost se računa poređenjem kategorije, materijala, cijene i prosječne ocjene proizvoda, a u nastavku je opisano kako sistem radi.

Putanja do source code-a recommender sistema:

StoneCarveManager.Services/Services/RecommenderService.cs

1. Priprema podataka

Prilikom poziva metode GetRecommendedProductsAsync, iz baze podataka se dohvaćaju:

- Ciljni proizvod (onaj koji korisnik trenutno gleda) sa svim relacijama (slike, recenzije, kategorija, materijal),
- Svi kandidat-proizvodi — samo oni koji su u stanju „active" (vidljivi korisnicima), isključujući sam ciljni proizvod.

```
2 references
public async Task<List<ProductResponse>> GetRecommendedProductsAsync(int productId, int count =
{
    var targetProduct = await _context.Products
        .Include(p => p.Images)
        .Include(p => p.Reviews)
        .Include(p => p.Category)
        .Include(p => p.Material)
        .FirstOrDefaultAsync(p => p.Id == productId, cancellationToken);

    if (targetProduct == null)
        throw new KeyNotFoundException($"Product with ID {productId} not found.");

    // Only recommend from active products (visible to users)
    var candidates = await _context.Products
        .Include(p => p.Images)
        .Include(p => p.Reviews)
        .Include(p => p.Category)
        .Include(p => p.Material)
        .Where(p => p.Id != productId && p.ProductState == "active")
        .ToListAsync(cancellationToken);

    if (candidates.Count == 0)
        return new List<ProductResponse>();
```

Iz svih proizvoda (kandidata + ciljnog) se zatim grade dvije referentne liste:

- allCategoryIds — lista svih jedinstvenih ID-ova kategorija,
- allMaterialIds — lista svih jedinstvenih ID-ova materijala.

Ove liste služe kao rječnik (vocabulary) za kreiranje one-hot vektora svake dimenzije.

```
// Build vocabulary for one-hot encoding
var allCategoryIds = candidates
    .Select(p => p.CategoryId)
    .Append(targetProduct.CategoryId)
    .Where(id => id.HasValue)
    .Select(id => id!.Value)
    .Distinct()
    .OrderBy(id => id)
    .ToList();

var allMaterialIds = candidates
    .Select(p => p.MaterialId)
    .Append(targetProduct.MaterialId)
    .Where(id => id.HasValue)
    .Select(id => id!.Value)
    .Distinct()
    .OrderBy(id => id)
    .ToList();

// Determine price range for normalization
var allPrices = candidates.Select(p => (float)p.Price).Append((float)targetProduct.Price).ToList();
float minPrice = allPrices.Min();
float maxPrice = allPrices.Max();
```

Dodatno se određuje i raspon cijena (minPrice, maxPrice) za potrebe normalizacije.

2. Pretvaranje proizvoda u feature vektor

Da bi se mogao mjeriti stepen sličnosti između dva proizvoda, svaki proizvod se pretvara u feature vektor koji se sastoji od četiri komponente:

```
    var targetVector = BuildFeatureVector(targetProduct, allCategoryIds, allMaterialIds, minPrice, maxPrice);

    var scored = new List<(Product product, float score)>();

    foreach (var candidate in candidates)
    {
        var vector = BuildFeatureVector(candidate, allCategoryIds, allMaterialIds, minPrice, maxPrice);
        float score = CosineSimilarity(targetVector, vector);
        scored.Add((candidate, score));
    }

    var recommended = scored
        .OrderByDescending(x => x.score)
        .Take(count)
        .Select(x => _mapper.Map<ProductResponse>(x.product))
        .ToList();

    return recommended;
}
```

a) One-hot kodiranje kategorije

Kreira se vektor duljine jednake broju jedinstvenih kategorija. Ako proizvod pripada određenoj kategoriji, na odgovarajućem indeksu vektora se postavlja vrijednost 1, a na svim ostalim 0.

Primjer: Ako postoje kategorije [Fontane, Klupe, Stupovi] i proizvod pripada kategoriji „Klupe", vektor je: [0, 1, 0]

```
2 references
private float[] OneHotEncode(int? value, List<int> vocabulary)
{
    var vector = new float[vocabulary.Count];
    if (value.HasValue)
    {
        int index = vocabulary.IndexOf(value.Value);
        if (index >= 0)
            vector[index] = 1f;
    }
    return vector;
}
```

b) One-hot kodiranje materijala

Na isti način se kodira i materijal. Ako proizvod koristi određeni materijal (npr. „Bijeli mramor"), na njegovom indeksu u vektoru stoji 1.

Primjer: Ako postoje materijali [Bijeli mramor, Travertin, Vapnenac] i proizvod je od travertina, vektor je: [0, 1, 0]

c) Normalizirana cijena

Cijena proizvoda se normalizira u raspon [0, 1] koristeći min-max normalizaciju:

normalizirana_cijena = (cijena − min_cijena) / (max_cijena − min_cijena)

d) Normalizirana prosječna ocjena

Prosječna ocjena recenzija se normalizira u raspon [0, 1] dijeleći sa maksimalnom mogućom ocjenom (5.0):

normalizirana_ocjena = prosječna_ocjena / 5.0

Ako proizvod nema recenzija, ocjena se postavlja na 0.

Konačni feature vektor dobije se spajanjem svih komponenti:

[...kategorija_one_hot, ...materijal_one_hot, normalizirana_cijena, normalizirana_ocjena]

```
2 references
private float NormalizeValue(float value, float min, float max)
{
    if (Math.Abs(max - min) < 1e-5f)
        return 0f;
    return (value - min) / (max - min);
}
```

3. Izračunavanje sličnosti (Cosine Similarity)

Za poređenje proizvoda koristi se kosinusna sličnost (Cosine Similarity), po formuli:

$$\text{similarity} = \frac{A \cdot B}{\|A\| \times \|B\|}$$

Gdje je:

- $A \cdot B$ — skalarni produkt dvaju vektora (dot product),
- $\|A\|$ i $\|B\|$ — euklidske norme (magnitude) vektora.

Rezultat je vrijednost između 0 i 1:

- 1 — proizvodi su potpuno identični po feature vektoru,
- 0 — proizvodi nemaju nikakvu sličnost.

```
1 reference
private float CosineSimilarity(float[] v1, float[] v2)
{
    if (v1.Length != v2.Length)
        throw new InvalidOperationException("Feature vectors must have the same length.");

    float dot = 0f;
    float normA = 0f;
    float normB = 0f;

    for (int i = 0; i < v1.Length; i++)
    {
        dot += v1[i] * v2[i];
        normA += v1[i] * v1[i];
        normB += v2[i] * v2[i];
    }

    return dot / ((float)(Math.Sqrt(normA) * Math.Sqrt(normB)) + 1e-5f);
}
```

4. Proces preporuke

Kada korisnik odabere neki proizvod (ciljni proizvod):

1. Sistem dohvati ciljni proizvod i sve aktivne kandidat-proizvode iz baze.

2. Gradi referentne liste kategorija i materijala (vocabulary).

3. Mapira ciljni proizvod u feature vektor.

4. Zatim prolazi kroz sve kandidat-proizvode i računa kosinusnu sličnost između ciljnog proizvoda i svakog od njih.

5. Dobijeni rezultati se sortiraju po opadajućem redoslijedu sličnosti.

6. Kao preporuka se uzima top N proizvoda (u aplikaciji po defaultu N = 6).

```
public async Task<List<ProductResponse>> GetRecommendedProductsAsync(int productId, int count = 6, CancellationToken cancell
{
    var targetProduct = await _context.Products
        .Include(p => p.Images)
        .Include(p => p.Reviews)
        .Include(p => p.Category)
        .Include(p => p.Material)
        .FirstOrDefaultAsync(p => p.Id == productId, cancellationToken);

    if (targetProduct == null)
        throw new KeyNotFoundException($"Product with ID {productId} not found.");

    // Only recommend from active products (visible to users)
    var candidates = await _context.Products
        .Include(p => p.Images)
        .Include(p => p.Reviews)
        .Include(p => p.Category)
        .Include(p => p.Material)
        .Where(p => p.Id != productId && p.ProductState == "active")
        .ToListAsync(cancellationToken);

    if (candidates.Count == 0)
        return new List<ProductResponse>();

    // Build vocabulary for one-hot encoding
    var allCategoryIds = candidates
        .Select(p => p.CategoryId)
        .Append(targetProduct.CategoryId)
        .Where(id => id.HasValue)
        .Select(id => id!.Value)
        .Distinct()
        .OrderBy(id => id)
        .ToList();

    var allMaterialIds = candidates
        .Select(p => p.MaterialId)
        .Append(targetProduct.MaterialId)
        .Where(id => id.HasValue)
        .Select(id => id!.Value)
        .Distinct()
        .OrderBy(id => id)
        .ToList();
```

5. Primjer rada sistema

Zamislimo da korisnik gleda proizvod „Kamena klupa za vrt":

- Kategorija: Klupe
- Materijal: Bijeli mramor

- Cijena: 450 KM
- Prosječna ocjena: 4.5

Kada sistem poredi „Kamenu klupu" sa drugim proizvodom, npr. „Kameni stup — bijeli mramor":

- Kategorija: Stupovi
- Materijal: Bijeli mramor
- Cijena: 380 KM
- Prosječna ocjena: 4.2

Vektori oba proizvoda imaju poklapanje u materijalu (Bijeli mramor), slične cijene i slične ocjene. Zato će kosinusna sličnost biti relativno visoka, pa će „Kameni stup" vjerovatno biti u listi preporučenih proizvoda.

Ako se „Kamena klupa" poredi sa proizvodom „Mala fontana — travertin":

- Kategorija: Fontane
- Materijal: Travertin
- Cijena: 1200 KM

Pošto nema zajedničke kategorije ni materijala, a cijena je značajno drugačija, sličnost će biti blizu 0, i fontana neće biti visoko rangirana u preporukama.

6. Frontend — prikaz preporuka

Putanja do fajla na FE gdje je pozvan i prikazan recommender sistem:

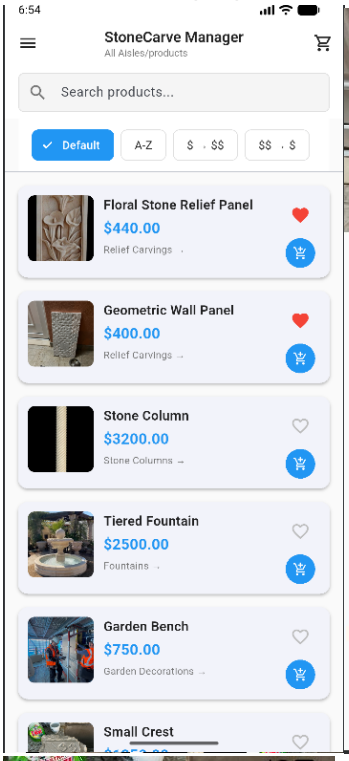
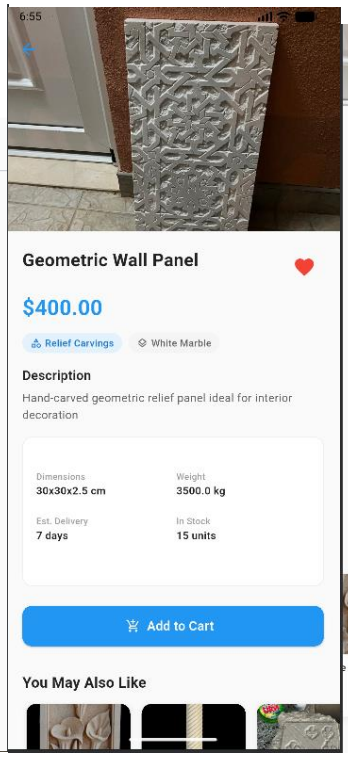
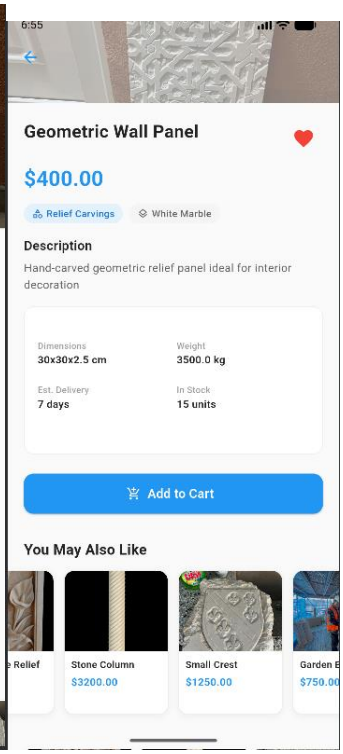
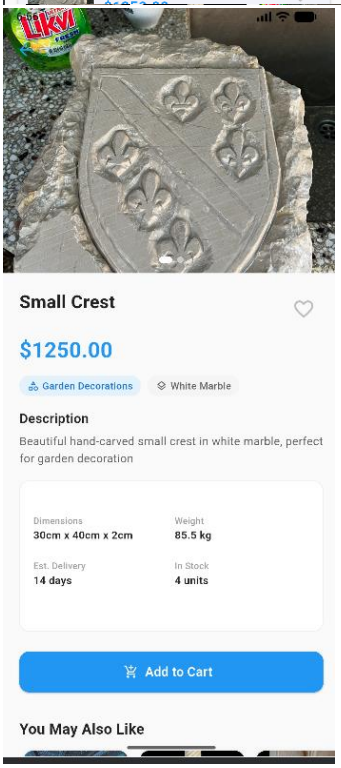
UI/stonecarve_manager_mobile/lib/screens/mobile/product_detail_screen.dart

API endpoint: GET /api/Product/{id}/recommendations?count=6

Koraci do lociranja sekcije za preporuku proizvoda u mobilnoj aplikaciji:

1. Prijaviti se na mobilnu aplikaciju sa korisničkim kredencijalima (kredencijale vidjeti u README.md fajlu).

2. Otvoriti katalog proizvoda i odabrati bilo koji proizvod.

3. Skrolati do kraja stranice s detaljima proizvoda, gdje će pod sekcijom „You May Also Like" biti prikazano do 6 proizvoda najsličnijih odabranom proizvodu.

4. Klikom na preporučeni proizvod otvara se novi detalj tog proizvoda sa vlastitim preporukama.